

AI TRAINING – DAY 4 ASSIGNMENT

Python Conditional Logic: Decide → Compare → Combine → Nest

Practical + MCQ + Logic Tracing + Debugging + 3 Daily Programs

Level	Beginner → Medium
Suggested Time	3–4 hours
Core Topics	if, elif, else, comparison operators, logical operators, nested conditions
Daily Deliverable	3 completed conditional programs
Main Focus	Learn to design and trace decision logic

Learning Goals

- Understand how Python makes decisions using if, elif and else.
- Use comparison operators correctly: ==, !=, >, <, >= and <=.
- Combine conditions using and, or and not.
- Understand when and why nested conditions are useful.
- Trace a program and predict which branch will execute.
- Identify common conditional-logic errors.
- Build three practical programs independently.
- Explain the logic in your own words.

Assignment Rules

- Write the logic yourself before asking AI for help.
- Before running, predict the output for at least 2 inputs.
- Use meaningful variable names.
- Test boundary values such as 0, 18, 35, 50, 60, 75 and 90 where relevant.
- Avoid giant, unreadable conditions.
- Document Input → Conditions → Output.
- Record one bug you faced and explain how you fixed it.

Part A – Understand the Decision Structure

Structure	Meaning	Your Task
if	Runs a block when a condition is True.	Use when you have one condition to check
if / else	Chooses between two paths.	If the condition is True run if block Otherwise run else block Ex:- find the largest number between two numbers
if / elif / else	Chooses among multiple possible paths.	Use when you need to check more than two conditions Ex:-find the largest number between 3 numbers
Nested if	A decision inside another decision.	When after one condition is True,you need to check another condition inside it

Part B – Comparison Operators Lab

Predict True or False before running:

- `10 > 5` ----->true
- `10 < 5` ----->false
- `10 == 10` -----true
- `10 != 10` ----->false
- `25 >= 25` ----->true
- `24 <= 20` ----->false
- `'Python' == 'python'` ----->false
- `'AI' != 'ML'` ----->true
- `100 == '100'` ----->false

Explain especially why `'Python' == 'python'` and `100 == '100'` produce their results.

Because python lag is a case sensitive .where P and p have difference.

And the 100 is an integer and '100' is an string

Part C – Logical Operators Lab

Predict the result:

- `True and True` -----> true
- `True and False` ----->false
- `False and True` ----->false
- `False and False` ----->false
- `True or False` ----->true
- `False or False` ----->false
- `not True` ----->true
- `not False` ----->true

Then create your own real-life and Python examples for AND, OR and NOT.

AND

I can sleep when I finish my assignment and push assignment into git

OR

i can sleep when I finish my assignment or the time is 8:00

NOT

Not I am try for job

Part D – Decision Tracing

Do not run these first. Predict the output, then test your answer.

```

1)
age = 20
if age >= 18:
    print("Adult")
else:
    print("Minor")----->output:-Adult

2)
marks = 82
if marks >= 90:
    print("A")
elif marks >= 75:
    print("B")
else:
    print("C")----->output:-B

3)
age = 20
has_id = True
if age >= 18 and has_id:
    print("Allowed")
else:
    print("Not allowed")----->output:- Allowed

4)
amount = 5000
if amount >= 10000:
    print("High")
elif amount >= 5000:
    print("Medium")
else:
    print("Low")----->output:-Low

5)
username = "admin"
password = "1234"

if username == "admin":
    if password == "1234":
        print("Login successful")
    else:
        print("Wrong password")
else:
    print("Unknown user")----->output:- Login successful

```

Part E – MCQs: Basic

1. Which keyword starts a condition?

A) if B) when C) check D) condition

2. Which keyword handles another condition after if?

A) else-if **B) elif** C) otherwise D) next

3. Which keyword handles the remaining case?

A) default **B) else** C) final D) otherwise

4. Which operator checks equality?

A) = **B) ==** C) === D) equals

5. What does > mean?

A) less than B) equal **C) greater than** D) not equal

6. Which operator means AND?

A) & **B) and** C) && D) plus

7. Which operator means OR?

A) or B) || C) | D) either

8. Which operator reverses a Boolean result?

A) reverse **B) not** C) ! D) opposite

9. In a single if/elif/else chain, how many else blocks are normally used?

A) 0 only B) exactly one C) many **D) two**

10. Can an if statement exist inside another if statement?

A) No **B) Yes** C) Only in loops D) Only in functions

Part F – MCQs: Medium

1. If 10 > 5, what is printed by if 10 > 5: print('Yes')?

A) Yes B) No C) Error D) True

2. Which is a correct age condition?

A) if age >= 18: B) if age => 18: C) if age = 18: D) if age greater 18:

3. In an if/elif chain, what happens after the first True condition?

A) All branches run B) **The first True branch runs and the chain stops** C) else always runs D) Program restarts

4. What is 10 > 5 and 2 > 8?

A) True **B) False** C) 10 D) Error

5. What is 10 > 5 or 2 > 8?

A) True B) False C) 8 D) Error

6. What is not(10 > 5)? A) True B) False C) None D) Error

7. Which represents age between 18 and 60 inclusive?

A) age >= 18 and age <= 60 B) age >= 18 or age <= 60 C) age > 18 and age > 60 D) age < 18 and age > 60

8. What is a nested condition?

A) A loop inside a string **B) A condition inside another condition** C) Two variables D) A function call

9. Nested conditions are useful when:

A) The second decision depends on the first B) You want no conditions C) Only for printing D) Only for loops

10. A common grade-boundary error is:

A) Using incorrect >= or > conditions B) Using print C) Using variables D) Using input

Part G – MCQs: Thinking

1. Why does elif order matter?

A) Python checks from top to bottom and stops at the first True branch B) It does not matter C) It sorts conditions D) It runs randomly

2. If A >= 90 and B >= 75, which should be checked first?

A) >= 75 **B) >= 90** C) < 75 D) == 90

3. Adult AND has ID can be written as:

A) age >= 18 and has_id B) age >= 18 or has_id C) age <= 18 and has_id D) not age

4. 'Not a student' can be represented by:

A) is_student B) not is_student C) is_student == True D) student not

5. A menu option 9 should be:

A) Ignored **B) Handled explicitly or rejected as invalid** C) Always option 1 D) Automatically accepted

6. Why test boundary values?

A) To find mistakes around >=, >, <= and < B) To make code longer C) To avoid input D) To remove variables

7. Why is age >= 18 or age <= 60 usually wrong for a range?

A) It is true for almost every number B) It always returns False C) or cannot be used D) Python does not support ranges

8. When is nested if more readable?

A) When the second decision depends on the first B) Always C) Never D) Only with strings

9. If many branches become hard to read, what should you do?

A) Make one giant condition **B) Break logic into smaller understandable steps** C) Remove tests D) Use random conditions

10. A grade calculator should test:

A) Only 100 **B) Normal, boundary and invalid cases** C) Only one mark D) No invalid input

Part H – Daily Deliverable: 3 Conditional Programs

Program 1 – Grade Calculator

- Ask for student name and marks.
- Validate marks from 0 to 100.

- Use if/elif/else with at least 5 grade categories.
- Display the grade and a meaningful message.
- Test 90, 75, 60, 50 and 0.
- Test invalid values such as -5 and 110.

```
name=input("Enter student Name : ")
marks=int(input("Enter marks :"))
grade=''
if marks < 0 or marks > 100:
    print("Invalid marks.")
else:
    if marks>=90:
        grade="A"
    elif marks >=75:
        grade="B"
    elif marks >=60:
        grade="C"
    elif marks >=50:
        grade="D"
    else:
        grade="F"
print(f" {name} got {marks} marks grade is {grade} ")
```

Output:-

Enter student Name : pavithra
Enter marks :78
pavithra got 78 marks grade is B

Program 2 – Eligibility Checker

Choose one realistic scenario: scholarship, course admission, job interview, driving eligibility, loan/application, or gym membership.

- Use at least 3 inputs.
- Use at least one and condition.
- Use at least one or condition.
- Create at least 3 possible outcomes.
- Write your eligibility rules clearly before coding.
- Test a qualifying case, a non-qualifying case and a boundary case.

```

age=int(input("Enter the age  of the Student:"))
Percentage=float(input("Enter your percentage:"))
Backlog=int(input("enter your backlogs :"))
if age>=18 and age<=24:
    if Percentage>= 75 and Backlog ==0:
        print(" you are Eligible")
    elif Backlog > 2 or Percentage > 60:
        print("you are not eligible")
    else:
        print("you are not eligible")
else:
    print("you are not eligible")

```

OUTPUT1:

Enter the age of the Student:20
 Enter your percentage:58
 enter your backlogs :1
 you are not eligible

OUTPUT2:

Enter the age of the Student:17
 Enter your percentage:79
 enter your backlogs :0
 you are not eligible

Program 3 – Simple Menu

- Create at least 4 menu choices.
- Ask the user for a choice.
- Use if/elif/else.
- Handle invalid choices.
- Use at least one nested if.
- Suggested project: Banking Menu — Check Balance, Deposit, Withdraw, Exit. For withdrawal, use a nested condition to check sufficient balance.
- Test every option and at least one invalid option.

```

int_balance=1000
balance=float(int_balance)
option=int(input("Enter the option :"))
if option == 1:

```

```

    print(f"your balance is {balance}")
elif option ==2:
    deposit=int(input("Enter the amount to deposite :"))
    balance=balance+deposit
    print(f"successfully add your amount \n your balance is {balance}")
elif option ==3:
    with_draw=int(input("Enter the amount to withdraw :"))
    balance=balance-with_draw
    print(f"successfully debited \n your balance is {balance}")
elif option ==4:
    print("thank you")
    exit()
else:
    print("invalid")

```

OUTPUT:

```

Enter the option :2
Enter the amount to deposite :4
successfully add your amount
your balance is 1004.0

```

Part I – Debugging Lab

For every bug: identify the problem, explain why it happens, fix it, and test it.

```

Bug 1
age = 18
if age = 18:
    print("Adult")

```

Hint: What is the difference between = and ==?

Correct code:-

```

age = 18
if age == 18:
    print("Adult")

```

Explanation:- = is an assignment oparetor and == is a comparing oparetor


```

Bug 2
marks = 85
if marks >= 50:
    print("Pass")
elif marks >= 80:
    print("Excellent")

```

Why does Excellent never print? Fix the order of conditions.

Correct code:-

```

marks = 85
if marks >= 80:
    print("Excellent")
elif marks >= 50:
    print("Pass")

```

Because 85 is greater than the 50 so if allows to print pass but 85 is greater than 80 actually it returns excellent

```

Bug 3
age = 25
if age >= 18 or age <= 60:
    print("Eligible")

```

Explain why this is wrong for an 18–60 range and correct it using and.

Correct code:-

```

age = 25
if age >= 18 and age <= 60:
    print("Eligible")

```

Because it allows all numbers if you use or when you use the and it allows only in between numbers

```

Bug 4
has_id = True
age = 17

if age >= 18 and has_id:
    print("Allowed")
else:
    print("Not allowed")

```

Predict the output and explain which part of the condition makes it False.

Correct code:-

```

has_id = True
age = 17

if age >= 18 and has_id:
    print("Allowed")
else:
    print("Not allowed")

```

It write the output is not allowed .when the id is wrong .but we write it always true.

```
Bug 5
choice = 2

if choice == 1:
    print("Deposit")
elif choice == 2:
    print("Withdraw")
elif choice == 2:
    print("Check Balance")
```

Why is the final branch unreachable? Redesign the menu conditions.

Correct code:-

```
choice = 2

if choice == 1:
    print("Deposit")
elif choice == 2:
    print("Withdraw")
elif choice == 3:
    print("Check Balance")
```

Because we right the 2 conditions same .when you enter choice 2 it always runs withdraw the python reads from top to bottom.

```
Bug 6
password = "1234"

if password == "1234":
    if password == "admin":
        print("Success")
    else:
        print("Wrong")
```

Explain the nested logic. Why can this never print Success? Redesign the login logic sensibly.

Correct code

```
password = "1234"

if password == "1234":
    if user == "admin":
        print("Success")
    else:
        print("Wrong")
```

Because we right the 2 conditions same .the python reads from top to bottom.password 1234 then it entry into the statement.when we write the password again it wrong so it print wrong always

Part J – Nested Conditions Challenge

Build a Login + Access Control program.

- Ask for username and password.

- If the username is correct, use a nested condition to check the password.
- If both are correct, ask whether the user is an admin.
- Show different messages for normal users and admins.
- Show separate messages for wrong username and wrong password.
- Test at least 5 combinations

```

Username="pavithra_4313"
password="pavi123"
enter_username=input("Enter the username:")
if enter_username==Username:
    enter_password=input("Enter the password :")
    if enter_password==password:
        User=input("enter the user:")
        if User=="admin":
            print("successfully login")
        else:
            print("verification is needed")
    else:
        print("Password is incorrect")
else:
    print("username is not matched")

```

oUTPUT:-

Enter the username:pavithra_4313
Enter the password :pavi123
enter the user:admin
successfully login

Enter the username:pavithra_4313
Enter the password :poi34
Password is incorrect

Enter the username:pavithra45667
username is not matched

Enter the username:pavithra_4313
Enter the password :pavi123
enter the user:user
verification is needed

Part K – Real-World Logic Challenge

Choose one: food-delivery charge calculator, movie-ticket pricing, college-attendance eligibility, ATM withdrawal, electricity-bill category, or mobile-data plan.

- Use at least 4 inputs or decision factors.
- Create at least 5 possible outcomes.
- Use comparison and logical operators.
- Use at least one nested condition.
- Draw a decision tree before coding.
- Test normal, boundary and invalid cases.

```
order_amount=int(input("enter order amount:"))
Distance=int(input("enter distance to your location :"))
membership=input("are you member in this app :(Yes/No):")
peak_time=input("are you busy at peak_time (yes/no):")
Delivery_charge=0
Total=0
discount=0
if order_amount>=1000:
    print("-> Free Delivery?")
    Delivery_charge=0
    print(f"delivery charge is {Delivery_charge}")
else:
    if Distance<=3:
        Delivery_charge=30
        print(f"delivery charge is {Delivery_charge}")
    elif Distance <=7:
        Delivery_charge=70
        print(f"delivery charge is {Delivery_charge}")
    elif Distance <=10:
        Delivery_charge=100
        print(f"delivery charge is {Delivery_charge}")
    else:
        print("not available for delivery")
if membership.lower()=="yes"and peak_time.lower()=="no":
```

```

    discount=20
elif membership=="yes"and peak_time=="yes":
    discount=10
else:
    discount=0
print(f"your discount {discount} and order amount {order_amount}")
print(f"the total bill = {order_amount+Delivery_charge-discount}")

```

Output:

```

enter order amount:34567
enter distance to your location :3
are you member in this app :(Yes/No):yes
are you busy at peak_time (yes/no):no
-> Free Delivery?
delivery charge is 0
your discount 20 and order amount 34567
the total bill = 34547

```

Part L – Draw a Decision Tree

Choose the Grade Calculator or Eligibility Checker.

- Start → Input → First decision → Branches → Next decisions → Final output.
- Your flowchart must match the code you finally submit.
- After coding, compare your flowchart with the actual program.

```

age=int(input("Enter the age of the Student:"))
Percentage=float(input("Enter your percentage:"))
Backlog=int(input("enter your backlogs :"))
if age>=18 and age<=24:
    if Percentage>= 75 and Backlog ==0:
        print(" you are Eligible")
    elif Backlog > 2 or Percentage > 60:
        print("you are not eligible")
    else:
        print("you are not eligible")
else:
    print("you are not eligible")

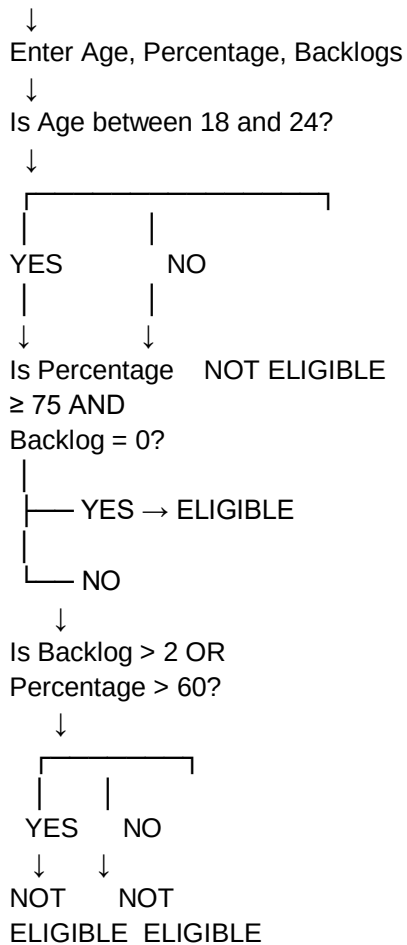
```

Output:

Enter the age of the Student:21
Enter your percentage:5
enter your backlogs :1
you are not eligible

FLOWCHAT:-

START

**Part M – Explain Your Code**

Why did you use if/elif/else instead of only if statements?

ANS: To check multiple conditions we use if/elif/else instead of if statement.

Where did you use comparison operators?

ANS: To compare two values or two variables we use comparison operators. some comparison operators are ==,>,>=,<,<=,! =.

Where did you use and/or/not?

ANS: I used and, or, and not to combine or compare conditions in a program.

Where did you use a nested condition?

ANS:I use nested condition, when one condition depends on another condition.The inner if condition depends on outer if condition.

Which boundary value caused the most thinking?

While writing code for marks, I faced boundary value in conditions like ,= or >=.

What bug did you encounter?

I encounter some bugs are indentation ,condition false,the condition always true.

How did you improve your first version?

Ans: I improved my first version by checking the conditions, correcting the indentation, and testing the code with different inputs.

Which condition would be hardest for another developer to understand?

ANS: To understand where to use < ,> <=,>=.

Part N – Self Check

Skill	yes	Notes
Basic if	yes	
if/elif/else	yes	
Using == correctly	yes	
>, <, >=, <=	yes	
and/or	yes	
not	yes	
Nested conditions	yes	
Tracing conditional programs	yes	
Invalid input/choices	yes	
Boundary testing	yes	
Completed 3 required programs	yes	
Can explain own logic	yes	

Submission Checklist

- Comparison operator experiments completed.
- Logical operator experiments completed.
- Decision tracing completed.
- All 30 MCQs answered.
- Grade Calculator completed.
- Eligibility Checker completed.
- Simple Menu completed.
- Debugging Lab completed.
- Nested Login + Access Control challenge completed.
- Decision tree submitted.
- Reflection and self-check completed.

Suggested Evaluation

Area	Weight
Concepts + MCQs	20%
3 Daily Programs	35%

Debugging	15%
Nested Challenge	15%
Testing + Explanation	15%

DAY 4 PRINCIPLE

*Don't just write conditions. Learn to think in decisions:
What is the input? What are the rules? What happens in every possible case?*